Multi-Agent Competition in SoccerTwos: Reward Shaping, Self-Play, and Imitation Learning

Bo Feng
Georgia Institute of Technology
bfeng66@gatech.edu

Frank Yang
Georgia Institute of Technology
frank.yang@gatech.edu

Abstract: We train PPO agents for the 2-vs-2 SoccerTwos environment, in which the default reward (a goal scored or conceded) is too sparse for the policy to gain traction from random initialisation. We add a six-signal reward-shaping wrapper (ball-approach, kick, offensive/defensive, time, team-separation), separate the policy and value networks, and mix the opponent pool 70% self-play / 30% CEIA baseline. The submitted agent reaches **90% (90/100) against the CEIA baseline** and **100% (20/20) against a random opponent**. A third agent warm-starts PPO from a behavioral-cloning policy that reaches 75.7% expert-action accuracy; it does not outperform random initialisation when fine-tuned against CEIA, which we trace to distribution shift, a cold-start critic, and the amplification of small action errors in adversarial play.

Keywords: Multi-Agent RL, PPO, Self-Play, Reward Shaping, Behavioral Cloning

1 Overview

SoccerTwos [1] is a 2-vs-2 physics-based soccer environment built on Unity ML-Agents in which goal events are sparse and the opponent is non-stationary under self-play. We submit three trained agents that share a PPO [2] backbone with Generalized Advantage Estimation [3] but differ in how the environment, opponent distribution, and initialization are handled: (i) **Agent 1 – PPO Self-Play (no shaping)**: a baseline trained with only the default game reward; (ii) **Agent 2 – PPO + Reward Shaping**, which is also our final submission, adds six dense rewards and trains with a 70% self-play / 30% CEIA opponent mixture using a separated policy/value network; (iii) **Agent 3 – BC + PPO Fine-tune** (bonus: imitation learning), which collects expert trajectories from an intermediate Agent 2 checkpoint, pretrains by behavioral cloning [4], and then fine-tunes against the CEIA baseline. All training uses Ray RLlib [5] v1.4.0 on the Georgia Tech PACE cluster.

2 Method

2.1 Algorithm and Architecture

Each agent is trained with PPO using the clipped surrogate objective

$$L^{\text{CLIP}}(\theta) = \mathbb{E}_t\Big[\min\big(r_t(\theta)\hat{A}_t,\ \text{clip}(r_t(\theta), 1-\epsilon, 1+\epsilon)\hat{A}_t\big)\Big], \tag{1}$$

where $r_t(\theta) = \pi_\theta(a_t|s_t)/\pi_{\theta_{\text{old}}}(a_t|s_t)$ and $\hat{A}_t$ is computed by GAE with $\lambda = 0.95$. The observation is the default 336-dimensional vector (ray casts plus relative positions/velocities of ball, goals, and the agent itself); the action is a MULTIDISCRETE(3,3,3) tuple controlling move/rotate/kick. Agents 1 and 3 use the Ray RLlib default `vf_share=True` (shared 256×256 MLP for policy and value); Agent 2 uses `vf_share=False` (two independent 256×256 MLPs); the separation prevents the value loss from dominating the shared gradient and was the single change that lifted our win rate from $\sim$78% to 90%.

2.2 Reward Shaping (Environment Modification)

The default game reward ($+1$ on a goal scored, -1 on a goal conceded) is too sparse to train from scratch. We wrap the multi-agent environment with a `RewardShapingWrapper` (`reward_shaping_ppo_agent/utils.py`) that adds six dense per-step rewards (Table 1). The hypothesis is that providing gradient signal *before* any goals are scored teaches basic ball-interaction skills (approach + kick), then offensive/defensive shaping orients the policy toward goal-directed play, and the separation reward discourages both teammates from clustering around the ball. All weights are kept small (max $|w| = 10^{-3}$) so that the original ± 1 goal signal still dominates as the policy improves.

Table 1: Per-step shaped reward signals added to the sparse goal reward.

Signal	Weight	Description
Approach	$+0.0002$	Velocity component toward the ball.
Kick	$+0.001$	Ball acceleration when the agent is within 1.5 units of the ball.
Offensive	$+0.0004$	Ball velocity toward the opponent goal.
Defensive	-0.001	Ball within 5 units of own goal.
Time penalty	-0.00002	Per-step cost to discourage stalling.
Separation	$+0.0001$	Distance between the two teammates (encourage spatial spread).

2.3 Training Strategy and Opponent Composition

We compare three opponent compositions: (a) pure self-play with a 4-snapshot archive that rotates whenever the team's reward exceeds 0.3; (b) team-vs-CEIA with 50% CEIA plus a 20-iteration cooldown to prevent over-frequent opponent updates; and (c) self-play 70% + CEIA 30%, inspired by Brandão et al. [6] who report high win rates in robotic soccer when self-play dominates the opponent pool but a fixed reference opponent is retained. This 70/30 mix gave us the highest CEIA win rate by combining strategy diversity from self-play with direct exposure to the evaluation opponent. Each iteration samples 20 000 environment steps across 7 parallel workers; training uses a learning-rate schedule from 3×10^{-4} down to 3×10^{-5}, an entropy bonus of 0.005, and gradient clipping at 0.5 (Table 2).

Table 2: Hyperparameters for the final reward-shaped self-play run (Agent 2).

Parameter	Value	Parameter	Value
`learning_rate`	$3 \times 10^{-4} \rightarrow 3 \times 10^{-5}$	`train_batch_size`	20 000
`clip_param`	0.2	`sgd_minibatch_size`	2 048
`entropy_coeff`	0.005	`num_sgd_iter`	10
λ (`GAE`)	0.95	`vf_loss_coeff`	0.5
`grad_clip`	0.5	`vf_share`	False (separated nets)
opponent mix	70% self-play / 30% CEIA	total steps	$\approx$ 30M

2.4 Bonus: Behavioral Cloning + PPO Fine-tune (Agent 3)

Training PPO from scratch against the full-strength CEIA opponent fails: the random initial policy is too weak to score, so the gradient signal collapses. We tried behavioral cloning as a warm start. (1) **Expert collection**: roll out our best-so-far checkpoint ($\approx 80\%$ win rate vs. CEIA) for 500 episodes, recording roughly 100 k (s, a) pairs. (2) **Supervised pretraining**: train a network with the same architecture as the PPO policy by cross-entropy on the three sub-actions (50 epochs, Adam, cosine learning-rate, batch 2 048; train/val split 90/10). (3) **PPO fine-tune**: inject the BC weights into a fresh PPO trainer via a custom `BCInitCallback`, then fine-tune against 100% CEIA with a lower learning rate (5×10^{-5}) and a higher entropy bonus (0.01) to compensate for the overly deterministic BC policy.

3 Experimental Results

Training curves. Figure 1 shows episode reward versus environment steps for the self-play baseline (Agent 1) and the reward-shaped self-play run (Agent 2). Reward shaping reaches the asymptotic level of the unshaped run $\sim$**3**$\times$ **faster** (around 1 M vs. 3-4 M environment steps); its plotted return is also higher because the shaped per-step signal is folded into the episode total.

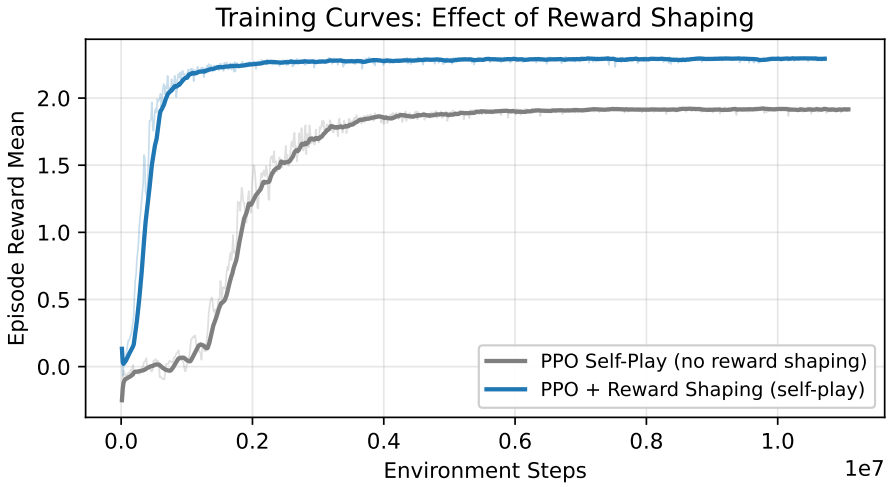

Figure 1: Episode reward vs. environment steps for the self-play baseline (Agent 1) and PPO with reward shaping (Agent 2). Light lines are raw values, bold lines are a 15-iteration moving average. Reward shaping reaches plateau roughly 3$\times$ sooner.

Performance comparison. Table 3 summarises win rates against the random and CEIA opponents. Win rate against CEIA was measured by 100 evaluation matches (`scripts/eval_vs_ceia.py`); the random-opponent rate was measured by 20 matches against `selfmade_random_agent` (`scripts/eval_vs_random.py`). The submitted agent (*PPO Reward Shaped + Self-Play 70% / CEIA 30%*, checkpoint-2300) reaches **90/100 = 90%** against CEIA and **20/20 = 100%** against the random opponent – both above the project's 9-of-10 target. The pure self-play agent oscillates between 46% and 62% during training, illustrating the instability of self-play without grounding in a fixed reference opponent. The Team-vs-CEIA variant (50% CEIA) reaches 68% but is dominated by the more diverse 70/30 mixture.

Table 3: Win rate of each agent across 100-match evaluation series (draws excluded from the percentages). Agent 3's reward is reported because it never converged to a positive win rate against CEIA.

Agent	vs. Random Agent	vs. CEIA Baseline
Agent 1: PPO Self-Play (no shaping)	–	46–62% (oscillating)
Agent 2 variant: PPO Team (50% CEIA)	–	68%
Agent 2 variant: Reward shaping (self-play)	–	85% (checkpoint-5750)
Agent 2: Reward Shaping + 70/30 (*submitted*)	**20/20 = 100%**	**90/100 = 90%**
Agent 3: BC + PPO fine-tune (bonus method)	–	reward $\approx$ 0.078 (not competitive)

Behavioral cloning ablation. The BC pretraining converged to 75.7% top-1 action-prediction accuracy (validation loss 0.579) in 12 minutes. After 12.5 M timesteps of PPO fine-tuning against 100% CEIA the episode reward was 0.078, indistinguishable from a randomly initialized PPO trained against CEIA from scratch (also 0.078 after a matched number of iterations).

4 Analysis and Discussion

Reward shaping accelerates convergence. Figure 1 shows that reward shaping converges about $3\times$ faster than the unshaped baseline. Each of the six dense signals provides a non-zero gradient even when no goal is scored, which breaks the chicken-and-egg problem of a sparse-reward MARL environment where both teams initially produce random play.

Self-play instability. Pure self-play oscillates between 15% and 62% win rate against CEIA. Two settings caused this: (1) with `entropy_coeff` $= 0$, policy entropy collapsed from $\sim$3.0 to $\sim$0.5 within 1 M steps, locking in narrow strategies that did not generalise beyond the self-play archive; (2) with `grad_clip` disabled, goal-event rewards produced large updates that erased prior learning. Setting `entropy_coeff` $= 0.005$ and `grad_clip` $= 0.5$ removed both failure modes. Switching to `vf_share=False` produced a further jump from $\sim$78% to 90% by stopping the value loss from dominating the shared gradient.

Opponent composition drives the CEIA win rate. Across our experiments the strongest lever on the CEIA win rate was the share of CEIA in the opponent pool, not the algorithm: 0% CEIA gave 46–62% (unstable), 50% CEIA reached 68%, and 70% self-play / 30% CEIA reached 90%. The 70/30 mix retains enough self-play for the agent to develop coordinated behaviour (e.g. passing, blocking) without over-fitting to a single fixed opponent, while still keeping the gradient signal anchored on the evaluation opponent.

Why behavioral cloning failed in this adversarial setting. 75.7% expert-action accuracy did not translate into any measurable advantage during PPO fine-tune. Three factors explain the gap. (a) *Distribution shift*: expert demonstrations were collected against a varied opponent, but fine-tuning runs against fixed CEIA, so the BC policy is queried on states it never saw and the residual 24.3% action error compounds. (b) *Value-function cold-start*: BC transfers only the policy network and leaves PPO's critic randomly initialised, so early advantage estimates are noisy and policy updates are unstable. (c) *Adversarial amplification*: each incorrect action hands the opponent a scoring opportunity, unlike single-agent or cooperative tasks where small errors are recoverable. The pattern matches the DAgger analysis [7] and the emergent-competition results of Bansal et al. [8]. A follow-up using DAgger or GAIL — which both re-query the expert on states the learner visits — would address (a) directly, though we have not run this experiment.

Code and reproducibility. All training scripts, the reward-shaping wrapper, the BC pipeline, and the evaluation utilities are available at
`https://github.com/Bo-Feng-1024/soccer-twos-starter`. The reward modification referenced in Table 1 is implemented in `reward_shaping_ppo_agent/utils.py` (lines 18-100), the BC pipeline in `train_bc_finetune.py` and `collect_expert_data.py`, and the self-play training loop in `train_ray_selfplay.py`.

References

[1] B. Oliveira. Soccer-twos: A multi-agent reinforcement learning environment based on Unity ML-Agents. `https://github.com/bryanoliveira/soccer-twos-env`, 2021.

[2] J. Schulman, F. Wolski, P. Dhariwal, A. Radford, and O. Klimov. Proximal policy optimization algorithms. *arXiv preprint arXiv:1707.06347*, 2017.

[3] J. Schulman, P. Moritz, S. Levine, M. Jordan, and P. Abbeel. High-dimensional continuous control using generalized advantage estimation. In *International Conference on Learning Representations (ICLR)*, 2016.

[4] D. A. Pomerleau. Efficient training of artificial neural networks for autonomous navigation. *Neural Computation*, 3(1):88–97, 1991.

[5] P. Moritz, R. Nishihara, S. Wang, A. Tumanov, R. Liaw, E. Liang, M. Elibol, Z. Yang, W. Paul, M. I. Jordan, and I. Stoica. Ray: A distributed framework for emerging AI applications. In *13th USENIX Symposium on Operating Systems Design and Implementation (OSDI)*, pages 561–577, 2018.

[6] B. Brandão, T. W. de Lima, A. Soares, L. Melo, and M. R. O. A. Maximo. Multiagent reinforcement learning for strategic decision making and control in robotic soccer through self-play. *IEEE Access*, 10:72628–72642, 2022. doi:10.1109/ACCESS.2022.3189021.

[7] S. Ross, G. Gordon, and D. Bagnell. A reduction of imitation learning and structured prediction to no-regret online learning. In *Proceedings of the Fourteenth International Conference on Artificial Intelligence and Statistics (AISTATS)*, pages 627–635, 2011.

[8] T. Bansal, J. Pachocki, S. Sidor, I. Sutskever, and I. Mordatch. Emergent complexity via multi-agent competition. In *International Conference on Learning Representations (ICLR)*, 2018.